

Serviço 1 — Users (NestJS + TypeORM + PostgreSQL)

Objetivo: criar, do zero, um microserviço **users** com NestJS e TypeORM usando Postgres (database-per-service), pronto para desenvolvimento colaborativo: Docker, migrações, DTOs, validação, Swagger, testes e health check, com fluxo Git/GitHub (issues, PRs, CI). Tudo aqui é **passo a passo** — basta seguir na ordem.

Visão arquitetural (rápida)

- **Microserviço:** `user-api` (porta 3000).
- **Banco dedicado:** `usersdb` (PostgreSQL, porta 5432).
- **Padrão:** database-per-service, consistência eventual p/ integrações futuras (eventos/outbox).
- **Borda HTTP:** REST com NestJS (Controller → Service → Repository/TypeORM).
- **Contratos:** DTOs de entrada/saída — *nunca* expor entidade diretamente.

Endpoints (CRUD mínimo): - `POST /users` — cria usuário (hash de senha, validação). - `GET /users` — paginação `?page=1&limit=10` + filtros simples (nome/email). - `GET /users/:id` — busca por `userId`. - `PUT /users/:id` — atualização idempotente. - `DELETE /users/:id` — remoção (hard delete neste MVP). - `GET /health` — health check do serviço e do banco. - Swagger: `/api/docs`.

0) Pré-requisitos

- **Node.js** LTS (18+), **npm** ou **yarn**.
- **Docker** e **Docker Compose**.
- **Git** + conta no **GitHub**.
- **VS Code** com extensões (recomendadas): ESLint, EditorConfig, Docker.

Se quiser isolar tudo, use Dev Containers. Não é obrigatório agora.

1) Repositório GitHub e fluxo de colaboração

1.1 Criar repositório

1. Acesse **GitHub** → **New Repository**.
2. Nome: `users-service` (padrão kebab-case).
3. Marque **Add .gitignore** → **Node** e **License** → **MIT** (ou escolha a sua).
4. Crie.

1.2 Clonar e primeiro commit

```
# substitua ORG/USER e URL conforme seu caso
git clone https://github.com/ORG/users-service.git
```

```
cd users-service

# configure sua identidade (se necessário)
git config user.name "Seu Nome"
git config user.email "seu@email"
```

1.3 Branches, PR e proteção

- Use `main` (estável) e `develop` (integração).
- Feature branches: `feat/users-crud`, `chore/docker`, etc.
- Ative **branch protection** para `main`: exigir PR + checks (CI) verdes.

1.4 Issues, Labels, Milestones e Project

- **Labels:** `feat`, `bug`, `docs`, `test`, `chore`, `infra`.
- **Milestones:** `M1 - Users MVP`, `M2 - Observabilidade`, ...
- **Project (Kanban):** *Todo* → *In progress* → *Review* → *Done*.

1.5 Templates e Contributing

Crie os arquivos abaixo:

```
.github/ISSUE_TEMPLATE/bug_report.md
.github/ISSUE_TEMPLATE/feature_request.md
.github/PULL_REQUEST_TEMPLATE.md
CODE_OF_CONDUCT.md
CONTRIBUTING.md
```

Sugestão de **PULL_REQUEST_TEMPLATE.md**:

```
## Descrição

## Checklist
- [ ] Testes passam (unit/e2e)
- [ ] Cobertura mantida ou ampliada
- [ ] Documentação/Swagger atualizado
- [ ] Sem alterações quebrando compatibilidade

## Issue
Closes #<id>
```

1.6 Conventional Commits

- `feat: criar endpoint POST /users`
- `fix: tratar conflito de email duplicado`
- `chore: configurar docker-compose`

Isso habilita changelog e versionamento semântico futuramente.

2) Bootstrap do projeto NestJS

2.1 Criar app

```
npm i -g @nestjs/cli
nest new user-api
# escolha npm ou yarn
mv user-api/* .
rm -rf user-api
```

2.2 Dependências

```
# runtime
npm i @nestjs/typeorm typeorm pg @nestjs/config @nestjs/swagger swagger-ui-
express class-validator class-transformer bcrypt @nestjs/terminus @nestjs/
throttler

# dev
npm i -D @types/bcrypt
```

2.3 Estrutura inicial (padrão sugerido)

```
src/
  main.ts
  app.module.ts
  config/
    configuration.ts
    validation.ts
  database/
    data-source.ts
    migrations/
  users/
    users.module.ts
    users.controller.ts
    users.service.ts
  dto/
    create-user.dto.ts
    update-user.dto.ts
    user-response.dto.ts
  entities/
    user.entity.ts
  mappers/
    user.mapper.ts
  health/
    health.controller.ts
```

2.4 Scripts no `package.json`

```
{
  "scripts": {
    "start": "nest start",
    "start:dev": "nest start --watch",
    "lint": "eslint .",
    "format": "prettier --write .",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:e2e": "jest --config ./test/jest-e2e.json",
    "migration:generate": "typeorm-ts-node-commonjs migration:generate src/database/migrations/auto -d src/database/data-source.ts",
    "migration:run": "typeorm-ts-node-commonjs migration:run -d src/database/data-source.ts",
    "migration:revert": "typeorm-ts-node-commonjs migration:revert -d src/database/data-source.ts"
  }
}
```

Observação: usamos o *CLI* do TypeORM em modo TS. Se preferir build antes, aponte para `dist`.

3) Docker + Docker Compose

3.1 `.dockerignore`

```
node_modules
npm-debug.log
Dockerfile
.git
.dist
.env
```

3.2 `Dockerfile`

```
# Desenvolvimento (hot-reload via bind mount)
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "run", "start:dev"]
```

Para produção, crie um build multi-stage com `npm run build` e `node dist/main.js`.

3.3 `docker-compose.yml`

```
version: '3.9'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: usersdb
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 10
    volumes:
      - pgdata:/var/lib/postgresql/data

  api:
    build: .
    command: npm run start:dev
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development
      - APP_PORT=3000
      - DB_HOST=db
      - DB_PORT=5432
      - DB_USER=postgres
      - DB_PASSWORD=postgres
      - DB_NAME=usersdb
      - DB_LOGGING=true
      - DB_SYNC=false
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./:/app

volumes:
  pgdata:
```

3.4 Variáveis de ambiente

Crie `.env.example` e **nunca** comite `.env` real:

```
NODE_ENV=development
APP_PORT=3000
DB_HOST=db
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=usersdb
DB_LOGGING=true
DB_SYNC=false
```

Execute:

```
docker compose up -d --build
```

4) Configuração do TypeORM e ConfigModule

4.1 src/config/configuration.ts

```
export default () => ({
  app: {
    port: parseInt(process.env.APP_PORT ?? '3000', 10),
  },
  db: {
    host: process.env.DB_HOST ?? 'localhost',
    port: parseInt(process.env.DB_PORT ?? '5432', 10),
    user: process.env.DB_USER ?? 'postgres',
    pass: process.env.DB_PASSWORD ?? 'postgres',
    name: process.env.DB_NAME ?? 'usersdb',
    logging: (process.env.DB_LOGGING ?? 'false') === 'true',
    sync: (process.env.DB_SYNC ?? 'false') === 'true',
  },
});
```

4.2 src/database/data-source.ts

```
import 'reflect-metadata';
import { DataSource } from 'typeorm';
import { config as load } from 'dotenv';
load();

export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '5432', 10),
  username: process.env.DB_USER,
```

```

password: process.env.DB_PASSWORD,
database: process.env.DB_NAME,
entities: [__dirname + '/../**/*.entity.{ts,js}'],
migrations: [__dirname + '/migrations/*.ts,js}],
logging: (process.env.DB_LOGGING ?? 'false') === 'true',
synchronize: false,
});

```

4.3 src/app.module.ts

```

import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import configuration from '../config/configuration';
import { TypeOrmModule } from '@nestjs/typeorm';
import { UsersModule } from '../users/users.module';
import { TerminusModule } from '@nestjs/terminus';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, load: [configuration] }),
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (cfg: ConfigService) => ({
        type: 'postgres',
        host: cfg.get('db.host'),
        port: cfg.get('db.port'),
        username: cfg.get('db.user'),
        password: cfg.get('db.pass'),
        database: cfg.get('db.name'),
        autoLoadEntities: true,
        logging: cfg.get('db.logging'),
        synchronize: false,
      }),
    }),
    TerminusModule,
    UsersModule,
  ],
})
export class AppModule {}

```

5) Entidade User e migração

5.1 src/users/entities/user.entity.ts

```

import {
  Entity, Column, PrimaryGeneratedColumn, CreateDateColumn, UpdateDateColumn,
  VersionColumn, Index

```

```

} from 'typeorm';

export enum UserRole { ADMIN = 'ADMIN', TEACHER = 'TEACHER', STUDENT =
'STUDENT' }

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  userId: string;

  @Column({ length: 120 })
  name: string;

  @Index('UX_users_email', { unique: true })
  @Column({ length: 160 })
  email: string;

  @Column({ select: false })
  passwordHash: string;

  @Column({ type: 'enum', enum: UserRole, default: UserRole.STUDENT })
  role: UserRole;

  @Column({ default: true })
  isActive: boolean;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;

  @VersionColumn()
  version: number;
}

```

5.2 Gerar migração

```

npm run migration:generate
npm run migration:run

```

Se algo mudar no schema, gere outra migração (nunca ligue `synchronize` em produção).

6) DTOs, validação e mapeamento

6.1 src/users/dto/create-user.dto.ts

```
import { IsEmail, IsEnum, IsNotEmpty, IsString, MinLength } from 'class-validator';
import { UserRole } from '../entities/user.entity';
import { ApiProperty } from '@nestjs/swagger';

export class CreateUserDto {
  @ApiProperty({ example: 'Ada Lovelace' })
  @IsString() @IsNotEmpty()
  name: string;

  @ApiProperty({ example: 'ada@exemplo.com' })
  @IsEmail()
  email: string;

  @ApiProperty({ minLength: 8 })
  @IsString() @MinLength(8)
  password: string;

  @ApiProperty({ enum: UserRole, required: false })
  @IsEnum(UserRole)
  role?: UserRole;
}
```

6.2 src/users/dto/update-user.dto.ts

```
import { PartialType } from '@nestjs/swagger';
import { CreateUserDto } from '../create-user.dto';
export class UpdateUserDto extends PartialType(CreateUserDto) {}
```

6.3 src/users/dto/user-response.dto.ts

```
import { ApiProperty } from '@nestjs/swagger';
import { UserRole } from '../entities/user.entity';

export class UserResponseDto {
  @ApiProperty()
  userId: string;
  @ApiProperty()
  name: string;
  @ApiProperty()
  email: string;
  @ApiProperty({ enum: UserRole })
  role: UserRole;
  @ApiProperty()
```

```

    isActive: boolean;
    @ApiProperty()
    createdAt: Date;
    @ApiProperty()
    updatedAt: Date;
    @ApiProperty()
    version: number;
  }

```

6.4 src/users/mappers/user.mapper.ts

```

import { User } from '../entities/user.entity';
import { UserResponseDto } from '../dto/user-response.dto';

export const toUserResponse = (u: User): UserResponseDto => ({
  userId: u.userId,
  name: u.name,
  email: u.email,
  role: u.role,
  isActive: u.isActive,
  createdAt: u.createdAt,
  updatedAt: u.updatedAt,
  version: u.version,
});

```

7) Módulo, Service e Controller

7.1 src/users/users.module.ts

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { User } from '../entities/user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}

```

7.2 src/users/users.service.ts

```
import { ConflictException, Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository, ILike } from 'typeorm';
import { User } from '../entities/user.entity';
import { CreateUserDto } from '../dto/create-user.dto';
import { UpdateUserDto } from '../dto/update-user.dto';
import * as bcrypt from 'bcrypt';

@Injectable()
export class UsersService {
  constructor(@InjectRepository(User) private repo: Repository<User>) {}

  async create(dto: CreateUserDto) {
    const exists = await this.repo.findOne({ where: { email: dto.email } });
    if (exists) throw new ConflictException('Email already in use');

    const passwordHash = await bcrypt.hash(dto.password, 10);
    const entity = this.repo.create({ ...dto, passwordHash });
    const saved = await this.repo.save(entity);
    return saved;
  }

  async findAll(page = 1, limit = 10, q?: string) {
    const skip = (page - 1) * limit;
    const where = q ? [{ name: ILike(`%${q}%`) }, { email: ILike(`%${q}%`) }] : {};
    const [items, total] = await this.repo.findAndCount({ where, skip, take: limit, order: { createdAt: 'DESC' } });
    return { items, total, page, limit };
  }

  async findOne(userId: string) {
    const found = await this.repo.findOne({ where: { userId } });
    if (!found) throw new NotFoundException('User not found');
    return found;
  }

  async update(userId: string, dto: UpdateUserDto) {
    const user = await this.findOne(userId);
    if (dto.email && dto.email !== user.email) {
      const exists = await this.repo.findOne({ where: { email: dto.email } });
      if (exists) throw new ConflictException('Email already in use');
    }
    if ((dto as any).password) {
      (dto as any).passwordHash = await bcrypt.hash((dto as any).password, 10);
    }
  }
}
```

```

        delete (dto as any).password;
    }
    await this.repo.update({ userId }, dto as any);
    return this.findOne(userId);
}

async remove(userId: string) {
    const user = await this.findOne(userId);
    await this.repo.delete({ userId: user.userId });
}
}

```

7.3 src/users/users.controller.ts

```

import { Controller, Get, Post, Body, Param, Query, Put, Delete,
ParseUUIDPipe } from '@nestjs/common';
import { UsersService } from './users.service';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
import { ApiTags, ApiQuery, ApiOkResponse, ApiCreatedResponse } from
'@nestjs/swagger';
import { UserResponseDto } from './dto/user-response.dto';
import { toUserResponse } from './mappers/user.mapper';

@ApiTags('users')
@Controller('users')
export class UsersController {
    constructor(private readonly service: UsersService) {}

    @Post()
    @ApiCreatedResponse({ type: UserResponseDto })
    async create(@Body() dto: CreateUserDto): Promise<UserResponseDto> {
        const saved = await this.service.create(dto);
        return toUserResponse(saved);
    }

    @Get()
    @ApiQuery({ name: 'page', required: false, example: 1 })
    @ApiQuery({ name: 'limit', required: false, example: 10 })
    @ApiQuery({ name: 'q', required: false, example: 'ada' })
    async findAll(
        @Query('page') page = 1,
        @Query('limit') limit = 10,
        @Query('q') q?: string,
    ) {
        const { items, total } = await this.service.findAll(Number(page),
Number(limit), q);
        return { items: items.map(toUserResponse), total };
    }
}

```

```

    @Get('/:id')
    @ApiResponse({ type: UserResponseDto })
    async findOne(@Param('id', new ParseUUIDPipe()) id: string):
    Promise<UserResponseDto> {
        return toUserResponse(await this.service.findOne(id));
    }

    @Put('/:id')
    @ApiResponse({ type: UserResponseDto })
    async update(
        @Param('id', new ParseUUIDPipe()) id: string,
        @Body() dto: UpdateUserDto,
    ): Promise<UserResponseDto> {
        return toUserResponse(await this.service.update(id, dto));
    }

    @Delete('/:id')
    async remove(@Param('id', new ParseUUIDPipe()) id: string) {
        await this.service.remove(id);
        return { success: true };
    }
}

```

8) main.ts: pipes globais, Swagger e segurança básica

```

import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { ValidationPipe } from '@nestjs/common';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { ConfigService } from '@nestjs/config';

async function bootstrap() {
    const app = await NestFactory.create(AppModule);
    app.enableCors();
    app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform:
    true }));

    const config = new DocumentBuilder()
        .setTitle('Users Service')
        .setDescription('API de usuários (CRUD)')
        .setVersion('1.0.0')
        .build();
    const document = SwaggerModule.createDocument(app, config);
    SwaggerModule.setup('api/docs', app, document);

    const cfg = app.get(ConfigService);
    const port = cfg.get('app.port') ?? 3000;
    await app.listen(port);
}

```

```
}  
bootstrap();
```

9) Health check

9.1 src/health/health.controller.ts

```
import { Controller, Get } from '@nestjs/common';  
import { HealthCheck, HealthCheckService, TypeOrmHealthIndicator } from  
  '@nestjs/terminus';  
  
@Controller('health')  
export class HealthController {  
  constructor(  
    private health: HealthCheckService,  
    private db: TypeOrmHealthIndicator,  
  ) {}  
  
  @Get()  
  @HealthCheck()  
  check() {  
    return this.health.check([  
      async () => this.db.pingCheck('database'),  
    ]);  
  }  
}
```

Não esqueça de exportar o `HealthController` no `AppModule` (ou criar `HealthModule`).

10) Testes

10.1 Unit (service, repositório mockado)

src/users/users.service.spec.ts (exemplo mínimo):

```
import { Test } from '@nestjs/testing';  
import { UsersService } from './users.service';  
import { getRepositoryToken } from '@nestjs/typeorm';  
import { User } from './entities/user.entity';  
  
describe('UsersService', () => {  
  let service: UsersService;  
  const repo = { findOne: jest.fn(), create: jest.fn(), save: jest.fn() };  
  // ...  
});
```

```

beforeEach(async () => {
  const module = await Test.createTestingModule({
    providers: [
      UsersService,
      { provide: getRepositoryToken(User), useValue: repo },
    ],
  }).compile();
  service = module.get(UsersService);
});

it('cria usuário com email único', async () => {
  repo.findOne.mockResolvedValue(null);
  repo.create.mockImplementation((x) => x);
  repo.save.mockImplementation(async (x) => ({ ...x, userId: 'uuid' }));
  const out = await service.create({ name: 'Ada', email: 'a@a', password:
'12345678' });
  expect(out.userId).toBeDefined();
});
});

```

10.2 E2E (opção rápida: apenas /health)

test/app.e2e-spec.ts:

```

import { Test } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import * as request from 'supertest';
import { AppModule } from '../src/app.module';

describe('App (e2e)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleFixture = await Test.createTestingModule({ imports:
[AppModule] }).compile();
    app = moduleFixture.createNestApplication();
    await app.init();
  });

  it('/health (GET)', async () => {
    const res = await request(app.getHttpServer()).get('/health');
    expect(res.status).toBe(200);
  });
});

```

Dica: para E2E completo com DB, use Testcontainers ou SQLite em memória — opcional neste MVP.

11) Rodando o projeto

```
# 1) Subir banco e API
docker compose up -d --build

# 2) Criar migrações e aplicar
npm run migration:generate
npm run migration:run

# 3) Acessar
curl http://localhost:3000/health
open http://localhost:3000/api/docs
```

Exemplos de requisições (HTTP/REST Client)

```
### Criar
POST http://localhost:3000/users
Content-Type: application/json

{
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "password": "supersegura",
  "role": "ADMIN"
}

### Listar
GET http://localhost:3000/users?page=1&limit=10&q=ada

### Buscar por ID
GET http://localhost:3000/users/{{userId}}

### Atualizar
PUT http://localhost:3000/users/{{userId}}
Content-Type: application/json

{ "name": "Ada L." }

### Remover
DELETE http://localhost:3000/users/{{userId}}
```

12) Checklist de aceite (MVP Users)

- [] Docker Compose com Postgres e API, health dos serviços ok.
- [] ConfigModule + TypeORM forRootAsync + migrations.
- [] Entidade `User` (UUID, email único, hash de senha, role, auditoria, version).
- [] DTOs de entrada/saída e mapeadores.

- [] Controller/Service com validação, conflitos e paginação.
- [] Swagger em `/api/docs` com schemas atualizados.
- [] Health check em `/health` (DB + app).
- [] Testes unitários mínimos e E2E de saúde.
- [] README com como rodar, scripts e exemplos.

13) CI (opcional mas recomendado)

Crie `.github/workflows/ci.yml`:

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --ci
```

Para E2E com DB, adicione um serviço Postgres no job ou use Testcontainers.

14) Troubleshooting (erros comuns)

- **ECONNREFUSED ao conectar no Postgres:** confira `DB_HOST=db` (não use `localhost` dentro do container). Aguarde health do DB (compose usa `depends_on` com `service_healthy`).
- **UnknownDependenciesException (Repository não resolvido):** verifique se o módulo importa `TypeOrmModule.forFeature([User])` e se o `UsersModule` foi importado no `AppModule`.
- **Swagger vazio:** confirme `SwaggerModule.setup('api/docs', ...)` após criar o documento e que os DTOs têm decorators `@ApiProperty`.
- **Email duplicado:** espere `409 Conflict` — é o comportamento correto do serviço (garantindo unicidade).

15) Roadmap de evolução

- **Soft delete** e audit trail.
- **Outbox** + evento `UserCreated` para integração com outros serviços.
- **Autenticação** (JWT) e RBAC com `UserRole`.
- **Observabilidade:** logger estruturado e métricas (Prometheus).
- **Paginação por cursor** e filtros avançados.

16) README (modelo)

Inclua no `README.md` algo como:

```
# Users Service (NestJS + TypeORM + PostgreSQL)

## Como rodar
```bash
docker compose up -d --build
npm run migration:run
open http://localhost:3000/api/docs
```

## Decisões

- Database-per-service.
- DTOs na borda, entidades internas.
- Migrations obrigatórias; `synchronize=false`.

---

## Conclusão

Seguindo os passos acima, você terá um **microserviço de usuários** funcional, containerizado, com migrações, documentação, testes e um fluxo de colaboração padronizado no GitHub. A partir dele, evoluímos para eventos/outbox, autenticação e integrações entre serviços.